

# Daily Scholar Papers Report — 2026-07-02

---

**1.1 · C-TO-RUST TRANSLATION · arXiv 2026** — six interpreters (6k–23k LoC) translated with 1–11 human interventions, 62–92% pass on held-out validation tests, and heap-overflow / use-after-free vulnerabilities in the C source disappear in the safe-Rust output

## **1.1 [Mostly Automatic Translation of Language Interpreters from C to Safe Rust](#) — Wang, Paulsen, Dodds, Kroening, Mathur, arXiv:2606.27122, 2026**

**Authors.** Bo Wang, Brian Paulsen, Joshua Dodds, Daniel Kroening, Umang Mathur. Kroening (Amazon Web Services / formerly Oxford, CBMC/ESBMC lineage) is the followed name; the rest are AWS Automated Reasoning collaborators plus academic co-authors — a strong verification-flavoured byline for a C-to-Rust migration paper.

**Venue.** arXiv:2606.27122 (cs.PL), submitted 20 June 2026. No conference target announced in the abstract; the closest fit given the empirical + verification angle is PLDI, OOPSLA, or FSE 2027.

**License.** arXiv non-exclusive distribution licence — figures may not be embedded verbatim; any structural depiction below is a Mermaid recreation.

**Problem.** C-to-Rust migration has become one of the most active applied-PL topics of the last two years (see C2Rust, Laertes, VERT, CRust, and the recent LLM-driven Vert/Rustify variants), but every existing pipeline hits the same wall: **safe** Rust requires satisfying the borrow checker, and real C programs — especially interpreter cores — encode ownership and aliasing patterns that no purely syntactic translator can lift. Interpreters matter because they (i) handle untrusted user input by design and (ii) concentrate the memory-unsafety risk of an entire language ecosystem in a single binary; a `muj s` / `lua-interp` / `duktape` vulnerability is a wide-blast-radius incident. The prior LLM-based translators were either evaluated on toy sizes or accepted `unsafe` blocks in the output, defeating the point.

### **Approach — feature-reduction milestones plus multi-agent validate-and-refine.**

Reboot’s core design idea is to reject the “translate the whole file in one shot” framing that every previous LLM-driven translator adopts. Instead:

- 1. Feature reduction.** Decompose the C program into a sequence of feature-restricted subsets: a version that only exercises arithmetic, then one that adds control flow, then one that adds heap allocation, then one that adds pointer aliasing, etc. Each subset is a *complete, testable* program. Translation proceeds from the smallest subset upward; every milestone must pass the (subsetting) test suite before the next feature is restored. This bounds the search space at every step and gives the multi-agent loop a hard, executable acceptance criterion.
- 2. Multi-agent orchestration.** A driver agent dispatches translation subtasks to worker coding agents, then routes their output through automated validators (Rust compiler + test suite + memory-safety sanitiser). Failures come back as structured feedback the driver can attribute to a specific milestone. Because milestones are small and validators are deterministic, the loop terminates in bounded time even though the coding agents themselves are unreliable.

**Headline numbers.** Six real-world interpreters translated: `muj s`, plus five others in the 6k–23k LoC range. Each translation required **1 to 11 brief user interventions** (the abstract’s phrase). All six pass

**100%** of the provided test suites; on **separately created validation tests never exposed to the system** they pass **62%–92%**. The security case study on `muj s` shows the C-side memory vulnerabilities (heap buffer overflow, use-after-free) are **eliminated** in the safe-Rust output — the borrow checker + Rust std collections rule them out by construction.

**Why this is Outstanding for this reader.** Three reasons converge. First, the *acceptance criterion* is deterministic (compilation, tests, sanitiser) — this is the same “make the LLM’s output validatable by a non-LLM tool” pattern the pipeline flagged as the deciding structural move in the 2026-06-30 Revelio report; two of the highest-impact papers this quarter share that architectural instinct. Second, interpreters are a *load-bearing* migration target — a working translation of `muj s` or `duktape` immediately becomes a candidate replacement in real embedded and browser-adjacent settings. Third, the reported 62–92% validation-set pass rate is honest — most prior LLM-translator papers report on the training tests only, and 62% on unseen tests is a much more useful number.

**Open questions.** The abstract does not disclose (i) which Rust toolchain features and lifetime patterns the system prefers (raw pointers via `NonNull ? Rc<RefCell<...>> ? arenas?`), (ii) whether the “brief user interventions” scale sub-linearly with LoC — 11 interventions on 23k lines is remarkable, but the same rate on 200k lines is 100+ interventions, and (iii) how well the pipeline handles C interpreters that JIT-compile (which requires `unsafe` for W-to-X memory) — none of the six interpreters listed in the abstract look like JIT-heavy targets, so this may be an out-of-scope condition. The full PDF is the natural next read.

**Verdict.** Outstanding — the strongest applied-PL result from the followed-researcher feed this month, with a deterministic acceptance criterion that generalises beyond translation, and a security case study that connects the PL work to a memory-safety incident category we care about.

## 2. Keep

---

**2.1 · TENSOR SUPEROPTIMIZATION · arXiv 2026** — SMT-over-unbounded-tensors soundness with algebraic-transformation propagation replaces hand-crafted rewrite rules; hardware-descriptor-driven tiling and operator/instruction fusion

**2.1 [Axon: A Synthesizing Superoptimizer for Tensor Programs](#) — Kothari, Zhu, Kroening, Sung, arXiv:2606.26344, 2026**

**Authors.** A. Kothari, S. Zhu, D. Kroening, C. Sung. Kroening on the byline again — this is the second Kroening-affiliated arXiv drop in the same 24 h window, both from the AWS Automated Reasoning line.

**Venue.** arXiv:2606.26344, submitted June 2026. Natural conference targets: PLDI, ASPLOS, MLSys 2027.

**License.** arXiv non-exclusive — no figure embedding.

**Problem.** Writing high-performance kernels for AI accelerators (TPU-class, tile-based DSAs, IREE targets) requires deep expertise in tiling, instruction selection, data layout, and operator fusion. Existing autotuning stacks (Ansor, TVM AutoTVM, Tenset, TC) rely on hand-crafted rewrite rules to explore semantically equivalent variants; new hardware means new rules. Superoptimisation for scalar ISA (Souper, STOKE, alive2) does not lift to tensor programs because the rewrite domain is unbounded (arbitrary shape and stride).

**Approach.** Axon combines three ideas:

- *Algebraic transformation discovery via propagation.* Operators are propagated through the computation graph, and semantic equivalence between variants is checked by **SMT over unbounded tensors** — no hand-crafted rewrite rules required. This is the paper’s main soundness contribution: the equivalence checker is total over the (parametric) input shape space, not a bounded set of examples.
- *Target ISA lowering.* Tensor operations are lowered to target-specific instructions with tiling configurations constrained by a hardware descriptor. The paper implies (but does not detail in the abstract) that the descriptor covers tile sizes, register file, and DMA characteristics.
- *Fusion.* Operators and instructions are fused to minimise memory traffic — the standard latency-hiding move on tile-based accelerators, but here integrated into the same synthesis loop as the equivalence check.

**Why Keep.** For a reader whose main axis is PL / program synthesis / verification, the interesting object is the *SMT-over-unbounded-tensors* soundness argument — this is a genuine technical contribution and, if the encoding is reusable, will show up in downstream systems. The abstract stops before revealing the empirical numbers, so headline speedups and coverage are not yet quotable; the full PDF is worth the follow-up read.

**Open questions.** (i) What’s the theory used for unbounded tensors — parametric arrays over integer indices? A dedicated tensor-logic SMT decision procedure? (ii) How does the equivalence checker time-out gracefully on shape-dependent transformations that cross an undecidable boundary? (iii) What’s the target hardware — the abstract says “tile-based AI accelerators” without naming a chip.

**2.2 · LLM STEERING · ICML 2026 workshop** — decouples calibration from decoding by casting steering as posterior sampling; conformal guarantees on generation-time steering rather than post-hoc filtering

## **2.2 [Conformal Steering of LLMs via Posterior Sampling](#) — Emmenegger, Olausson, Solar-Lezama, et al., ICML 2026 Statistical Frameworks for LLMs Workshop**

**Authors.** N. Emmenegger, T. X. Olausson, A. Solar-Lezama, et al. Solar-Lezama (MIT CSAIL) is the followed name; Olausson is a Solar-Lezama student whose earlier work on LLM-driven program synthesis (2023–2025) sits inside this reader’s watch-list.

**Venue.** ICML 2026 Statistical Frameworks for LLMs Workshop. Workshop venues score lower than the main track on the Stage-1 top-venue axis, but the followed-researcher override applies; the paper is worth a summary rather than a full deep-read here.

**License.** OpenReview default; figures not embedded.

**Problem.** LLM hallucination is now being attacked with conformal prediction, but existing conformal-LLM methods (Angelopoulos et al., Kumar et al., 2023–2025) operate in a *post-hoc filter* mode: generate a candidate, decide whether to keep it against a calibrated threshold. This wastes decoded tokens on candidates that will be rejected, and — more subtly — cannot steer the model *during* generation.

**Approach.** The paper casts steering as **posterior sampling**: given a prompt and a target property, define a distribution over completions conditioned on the property, then sample from it using an approximation that pairs a base LLM proposal with a calibrated conformal accept-reject step. The

“conformal” adjective in the title is doing real work — the guarantee is a coverage bound on the target property that holds under exchangeable calibration data, not a heuristic thresholding scheme.

**Why Keep.** The design pattern is portable. Program-synthesis workflows that use LLMs as proposal distributions for a downstream verifier (Solar-Lezama’s group has been building these for a decade) can plug this in wherever there’s a calibrable acceptance property — type correctness, spec conformance, test-pass rate. Whether the empirical numbers justify the theory is a follow-up question the workshop abstract does not fully answer.

**Open questions.** (i) What’s the calibration set — outputs from the same model, or from a stronger judge? (ii) How does compute cost compare to rejection sampling, per accepted token? (iii) Does the posterior-sampling framing extend to multi-step reasoning (chain-of-thought / tool-use) or does the exchangeability assumption break under agentic trajectories?

**2.3 · LLM PROGRAM REPAIR · arXiv 2026** — 7,745 SWE-bench agent traces + 3,000 end-to-end runs across Claude Code / Codex / OpenCode: average 8.8 test runs per task, late-stage executions win, Prohibited vs Unrestricted gap only 1.25 pp on SOTA commercial agents

**2.3 To Run or Not to Run: Analyzing the Cost-Effectiveness of Code Execution in LLM-Based Program Repair** — Lin, Zhu, Zhou, Wang, Sun, Yang, Lo, et al.,  
**arXiv:2606.26978, 2026**

**Authors.** Z. Lin, J. Zhu, M. Zhou, X. Wang, Z. Sun, R. Yang, D. Lo, et al. David Lo (SMU) is the followed name; the byline is his SMU-anchored software analytics group.

**Venue.** arXiv:2606.26978 (cs.SE), submitted June 2026. Natural targets: FSE, ICSE, ASE 2026/27.

**License.** arXiv non-exclusive — no figure embedding.

**Problem.** LLM-based agents for program repair (SWE-agent, Claude Code, Codex, OpenCode) have converged on a *generate-run-revise* loop: propose a patch, run the test suite, use the failure feedback to revise. Execution is expensive — compute, sandbox provisioning, and wall-clock latency all scale with test-run count — but its **incremental contribution** to resolve-rate has never been measured against a well-controlled counterfactual. Practitioners are running 5–20 tests per task on production infrastructure without knowing whether that number could safely drop.

**Approach.** A two-stage empirical study:

- *Descriptive stage.* Analyse **7,745 agent traces** scraped from SWE-bench leaderboard submissions. Extract per-task test-run counts, timing within the trajectory, and outcome.
- *Controlled stage.* Run **3,000 end-to-end repair attempts** across **200 SWE-bench instances** and **three agents (Claude Code, Codex, open-source OpenCode)** under **four execution paradigms** — Prohibited (no execution), Restricted, Unrestricted, and a fourth variant the abstract does not name. Compare resolve-rate and cost per paradigm.

**Headline numbers (from the abstract).**

- Code execution is used **across all agents and models**, averaging **8.8 test runs per task**; per-task frequency ranges **2 to 19**. Late-stage executions **consistently achieve higher success rates** than early-stage ones.

- On SOTA commercial agents, the resolve-rate gap between **Prohibited** and **Unrestricted** is only **1.25 percentage point**. The abstract cuts off before revealing whether the same holds for open-source OpenCode.

**Why Keep.** The 1.25 pp finding is a genuine falsification of a widely held prior: execution is the “obvious” ingredient that makes agentic repair work, and the community treats unrestricted sandbox access as table-stakes. If the finding holds up to scrutiny, execution’s true value is smaller than assumed and the industry can save significant sandbox / CI cost. For the reader’s own agentic tooling, the late-stage-executions-win observation is directly actionable — schedule executions toward the end of a trajectory, not the start.

**Open questions.** (i) The abstract does not disclose the fourth execution paradigm; (ii) 200 SWE-bench instances is a small sample — does the effect replicate on SWE-bench-Verified or the full 2,294-task set? (iii) The “SOTA commercial” carve-out on the 1.25 pp finding is suspicious — the gap is likely larger for weaker agents, which is what the abstract’s mention of OpenCode is probably hinting at.

**2.4 · REPO-LEVEL RAG · ACM TOSEM 2026** — repository-level retrieval-augmented generation for SE tasks; the paper’s angle is a snippet-retrieval formulation replacing full-file retrieval

## **2.4 [Streamlining Repository Tasks with Effective Snippet Retrieval](#) — Xu, Li, Xu, Jiang, Yao, Zhu, Xu, Di, et al., ACM Transactions on Software Engineering and Methodology, 2026**

**Authors.** T. Xu, C. Li, Z. Xu, Y. Jiang, Y. Yao, X. Zhu, F. Xu, P. Di, et al. Followed via Zhendong Su alert; Su is not on the author list — the alert fires because the paper cites Su’s group. The named byline maps to Nanjing / Tsinghua SE collaborators.

**Venue.** ACM TOSEM, DOI 10.1145/3821433, 2026. TOSEM is a top-tier archival journal on the SE side, so venue-quality is high; the paper qualifies for the Keep bucket on venue alone.

**License.** ACM DL — no figure embedding; recreations only.

**Problem.** Repository-level SE tasks (code completion, edit prediction, review, repair) are increasingly implemented as **repo-level retrieval-augmented generation (RLRAG)**: at inference time, a retriever pulls context from the repository into the LLM prompt. The dominant retrieval unit has been the *whole file* (or class), because it aligns with import boundaries. But whole-file retrieval wastes prompt budget on irrelevant parts of large files, and the “right” context for many tasks is a *snippet* — a function body, a slice of a config block, a specific test case.

**Approach.** The paper (from the abstract snippet) reformulates the retriever to return **snippets** rather than files. Concrete details of the retriever (tokenisation, chunking strategy, ranking model) are not visible in the Scholar snippet; the ACM DL landing page requires institutional access to read further.

**Why Keep.** RLRAG is a bread-and-butter topic for anyone building agentic repo tooling, and a TOSEM archival paper with a clean snippet-vs-file comparison is a useful reference. The strength of the paper will hinge on whether the snippet chunking is *task-adaptive* (different granularity for review vs completion) or *task-agnostic* — the former is the interesting research position, the latter is engineering.

**Open questions.** (i) Chunk-boundary strategy — AST-based, brace-based, sliding-window? (ii) Retrieval evaluation — MRR against a ground-truth snippet oracle, or downstream task success? (iii) How does snippet retrieval interact with agent memory / cross-turn context reuse?

### 3. Borderline High-Priority

---

**3.1 · ADVERSARIAL LLM DEFENSE · arXiv 2026** — genetic algorithm over 8 semantics-preserving obfuscations; up to 70% attack success rate inducing refuse/misinterpret on 7 LLMs including GPT-4o, DeepSeek, Qwen, Llama, Gemma

#### **3.1 [Acoda: Adversarial Code Obfuscation for Defending against LLM-based Analysis](#)** — Rao, Dong, Zhao, Li, Wang, arXiv:2606.11755, 2026

**Authors.** H. Rao, Z. Dong, Y. Zhao, H. Li, H. Wang. Not on the followed-researcher list; came through the Recommended articles alert. Affiliations not visible from the abstract snippet.

**Venue.** arXiv:2606.11755, 2026. No conference target announced.

**License.** arXiv non-exclusive — no figure embedding.

**Problem.** LLMs are being deployed to analyse third-party code for understanding, debugging, and vulnerability detection. That's a productivity win for defenders, but it's also an offensive capability against IP-protected code: an attacker can now reverse-engineer proprietary logic by pushing the source through a strong general-purpose LLM. Existing code-obfuscation techniques (control-flow flattening, identifier renaming, virtualisation) were designed against *static analysers*, not against LLMs whose semantic reasoning can see through many syntactic transformations.

**Approach.** Acoda is a **genetic-algorithm-driven adversarial obfuscator** targeting the LLM as the analyser. It:

1. Defines **8 semantics-preserving obfuscation methods** exploiting two LLM mechanisms — **safety alignment** (obfuscations that look adversarial trigger refusal) and **token-based information processing** (obfuscations that split semantic units across BPE token boundaries).
2. **Iteratively optimises** an obfuscation strategy per input via GA; fitness is measured by an **auxiliary LLM + four evaluation metrics** on how well the target LLM analyses the obfuscated code.

**Headline numbers.** Up to **70% attack success rate** (target LLM refuses or misinterprets) across **7 SOTA LLMs** — GPT-4o, DeepSeek, Qwen, Llama, Gemma. Strong cross-model transferability. Minimal runtime overhead. Semantics claimed to be preserved (the abstract cuts off before the details of the preservation check).

**Why Borderline.** The paper is interesting as a *defence* paper for IP-protection settings but the framing is uncomfortably close to *offensive* obfuscation-as-a-service — the same technique that hides proprietary code from a helpful LLM also hides malware from a security-analysis LLM. Any downstream adoption should be paired with a security-impact analysis. Methodologically the fitness-function design (auxiliary-LLM judge + four metrics) is reusable in adjacent LLM-adversarial settings, which is the main reason to keep it in the reading queue rather than skip.

**Open questions.** (i) Do the obfuscations survive automatic normalisation (formatting, renaming to canonical identifiers, LLM-guided deobfuscation)? (ii) Is the 70% ASR measured on natural analysis prompts, or on prompts specifically shaped to trigger refusal? (iii) How does the “safety alignment abuse” component interact with the target model's refusal rate on the *unobfuscated* baseline — some fraction of the 70% may be due to the base refusal rate, not obfuscation.



**3.2 · CONCOLIC TESTING · arXiv 2026** — LLM as bounded discovery oracle (not a solver replacement); +8.6 / +15.1 / +17.0 pp mean-line-coverage on synthetic / real-world / library targets at \$1.63 total API spend

### **3.2 ConcoLixir: Reactive LLM Discovery Oracles for Python Concolic Testing — Chen, Hong, Yu, arXiv:2606.26545, 2026**

**Authors.** D. Chen, C. D. Hong, F. Yu. Not on followed-researcher list; Recommended articles alert.

**Venue.** arXiv:2606.26545 (cs.SE), 2026.

**License.** arXiv non-exclusive — no figure embedding.

**Problem.** Concolic testing on Python is chronically limited by three failure modes: (i) *symbolic downgrade* — library calls consume a symbolic variable and return a concrete value, losing the constraint; (ii) *hard semantic operations* — regexes, checksums, parsers, encoders resist SMT modelling; (iii) *exploration plateau* — the search returns to already-covered paths and stalls.

**Approach — LLM as bounded discovery oracle.** The paper deliberately positions the LLM as a **discovery oracle**, not a replacement for the solver and not a correctness oracle. Three roles:

1. Generate initial **seed inputs** so the concolic search does not start from `None`.
2. When the **solver fails** on a hard constraint, propose a concrete input that satisfies it heuristically.
3. When **coverage stalls**, target **uncovered code** with LLM-generated candidates.

Every LLM candidate is then executed concolically — observed coverage and collected path constraints are the only signals that guide later exploration. The LLM never asserts a specification about the program's behaviour; it only produces inputs.

**Headline numbers.** Across synthetic, real-world, and library-heavy targets, ConcoLixir improves **mean line coverage** over the no-LLM baseline by **+8.6, +15.1, and +17.0 percentage points** respectively. The gains are strongest near **semantic barriers** (checksums, regexes) and **library boundaries** (where symbolic values downgrade). Total API spend for the full evaluation is **\$1.63**.

**Why Borderline (rather than Keep).** The methodological framing — LLM as bounded discovery oracle rather than solver replacement — is exactly the pattern several other 2026 papers in this reader's queue have converged on (Revelio's sanitizer-checked hypotheses, Reboot's validate-and-refine, and now this). The trend deserves to be tracked even when individual papers are workshop-scale. The two reservations that keep it out of Keep are (i) Python-only scope, and (ii) the coverage-only success metric — the paper does not (per abstract) report new bugs found, which is the harder ask for a testing tool.

**Open questions.** (i) Which LLM was used, and how does the ranking change with a cheaper model? (ii) What baseline concolic tester — CrossHair, PyExZ3, Klee-python-front-end? (iii) Does the "reactive" scheduling (only invoke the LLM on solver failure or coverage stall) generalise to fuzzers, or is it specific to path-constrained testing?

---

## Cross-Paper Synthesis

---

Two structural threads run through today's non-empty batch.

**Thread 1 — deterministic acceptance criteria for LLM outputs.** Three of the six deep-read papers — Reboot (1.1), ConcoLixir (3.2), and, if we allow the extension, Conformal Steering (2.2) — share the same architectural instinct: put a *non-LLM validator* in the loop and let the LLM propose against it. Reboot uses the Rust compiler plus test suite plus memory sanitiser as the acceptance oracle. ConcoLixir uses the concolic engine — observed coverage and collected path constraints — as the sole signal that guides continued exploration. Conformal steering uses the calibrated conformal accept-reject step to bound the target property. Compared with the 2026-06-30 Revelio report (sanitizer-checked Proof-of-Vulnerability) and 2026-06-29's THEMIS (grey-box fuzzer as oracle for WordPress plugins), a five-paper streak of “deterministic-oracle-in-the-loop” has now emerged in this reader's window. The corollary for own tooling: any agentic loop worth building this quarter should have a clearly identifiable non-LLM acceptance step, and the report where that step is missing should default to lower priority.

**Thread 2 — empirical falsification of default assumptions in LLM-agent stacks.** The David Lo group's “To Run or Not to Run” (2.3) reports a **1.25 pp** resolve-rate gap between Prohibited and Unrestricted execution on SOTA commercial agents — a plausible falsification of the widely held prior that unrestricted sandbox execution is table-stakes for agentic APR. This is a rare class of paper (empirical study designed to falsify a well-motivated assumption in a hot subfield) and is worth cross-referencing against the earlier trace-analysis papers of 2024–2025 (Yang et al.'s SWE-agent scaling laws, Xia et al.'s agentic-repair post-mortems). If the 1.25 pp holds up, the cost-optimal APR agent architecture may look very different from today's.

The two threads intersect in a useful way. If deterministic-oracle-in-the-loop is the architectural pattern that lets LLM outputs be trusted, then the specific *cost* of the oracle step (test execution, compilation, sanitiser run) becomes the top variable to tune — which is exactly what “To Run or Not to Run” is measuring. Reboot's milestone-based translation is an implicit answer to the same question in a different domain: run the sanitiser at every milestone (small, cheap oracle calls), not once at the end.

---

## Writing & Rationale Insights

---

- *Followed-researcher signal is the highest-quality slice of the feed.* Today four of the five Scholar alert threads came from the four followed accounts (Kroening, Solar-Lezama, David Lo, Zhendong Su); the fifth was the general `Recommended articles` batch. Of the followed-researcher papers, three cleared Stage-1 into Keep+ and one (Reboot) landed in Outstanding — a ~80% survival rate versus roughly 33% for the Recommended batch (2 kept out of 6). The list of followed accounts is doing meaningful work; expanding it deliberately (not reactively) is a lever worth pulling.
- *Two Kroening drops in one day is a pattern, not noise.* Both preprints are AWS Automated Reasoning outputs (Reboot on interpreter translation, Axon on tensor superoptimisation). AWS AR appears to be batching submissions — worth watching for the next batch and reading the accompanying blog / talks for framing.
- *Preference layer still cold on this-quarter authors.* Kroening does not have an entry in `.private/preferences.json` because he hasn't received a thumbs-up yet; the followed-



researcher override carried today. As feedback accumulates on Kroening-affiliated papers, the preference layer should start reinforcing the followed-researcher signal quantitatively.

- “*To Run or Not to Run*” is a *template*. Its two-stage design (large-N descriptive stage on public traces + medium-N controlled stage with counterfactual paradigms) is a reusable empirical template for other agentic-LLM subfields — worth naming and re-applying when planning own empirical studies.